A PROJECT REPORT ON

HACKAGON

A Web Platform for Hosting and Joining Online Coding Competitions

INDIVIDUAL CONTRIBUTION REPORT

Member 3: Money, Core and DevOps

Submitted in partial fulfilment of the requirements for the degree of
Bachelor of Technology in Computer Science and Engineering

Submitted by

Hemanth Singh Tanwar

(Member 3 of the HackAgon team: Hiten Sharma, Harsh Sharma, Hemanth Singh Tanwar, Arjun Suman, Akshat Rathore)

Under the guidance of
Er. Ram Babu Buri

Department of Computer Science and Engineering
Arya College of Engineering and IT, Jaipur
Session 2026-27

CERTIFICATE

This is to certify that the individual contribution report entitled "HackAgon: Individual Report of Member 3 (Money, Core and DevOps)", submitted by Hemanth Singh Tanwar, a student of the third year of Bachelor of Technology in Computer Science and Engineering at Arya College of Engineering and IT, Jaipur, is a record of his own work carried out during the session 2026-27 as a member of the HackAgon project team, under my guidance and supervision.

The work embodied in this report has not been submitted elsewhere for the award of any degree or diploma. The report is, in my opinion, worthy of consideration as part of the project submission.

Er. Ram Babu Buri	[Head of Department]
Project Guide	Head of Department

Date: _____

Place: Jaipur

DECLARATION

I, Hemanth Singh Tanwar, hereby declare that the individual contribution report entitled "HackAgon: Individual Report of Member 3 (Money, Core and DevOps)" is an original account of the work done by me as Member 3 of the HackAgon project team, under the guidance of Er. Ram Babu Buri.

I further declare that this report has been prepared from the HackAgon Project Report, the HackAgon Daily Work Log and the HackAgon Team Modules and 12-Week Plan, and has not been copied from any other source. Wherever the work of other team members is described, it is attributed to them by name or by member number.

Hemanth Singh Tanwar
Date: _____
Place: Jaipur

ACKNOWLEDGEMENT

I would like to thank everyone who helped me complete my part of the HackAgon project. I am deeply grateful to our project guide, Er. Ram Babu Buri, for steady direction, patient review and useful suggestions through every stage of the project. I thank the Head of the Department of Computer Science and Engineering and the faculty of Arya College of Engineering and IT, Jaipur, for the learning environment and the laboratory facilities.

I thank my teammates Hiten Sharma, Harsh Sharma, Arjun Suman and Akshat Rathore. The money and platform modules are useful only because their accounts, rooms, judge, admin tools, proctoring and screens were built in the same weeks and fitted together at the first real tournament. Their reviews of my pull requests, and mine of theirs, taught me as much about teamwork as about code.

I also thank the open-source community and the providers whose free tiers and documentation made this project possible: Java, Spring Boot, MySQL, TiDB Cloud, Render, GitHub and Razorpay's test mode. Finally, I thank my family and friends for their encouragement, and the friends who played the first small tournament.

Hemanth Singh Tanwar

ABSTRACT

HackAgon is a web application for hosting and joining online coding competitions, with solo, duo, squad and duel rooms, paid tournaments with a funded prize pool, automatic judging, proctoring and an administrator back office. Five students built it between 6 July and 6 October 2026. This report describes the contribution of Member 3, Hemanth Singh Tanwar, who owned Module 5, Tournaments and Money, and Module 6, Platform Core and DevOps.

For Module 5 the report covers the money tables and the append-only ledger, the Razorpay test account and money contract, host funding with a signed and idempotent webhook, registration that stays closed until the pool is funded, entry fees, two refund paths, payout details with OTP and account validation, the payout job with the 1% fee and the split by rank, and the ledger balance check. For Module 6 it covers the database decision, schema V1, the migration runner and repository layer, the protective layer of rate limits, validation, CORS and a single error handler, the security review, the load simulation up to 500 players, backups and a tested restore, and the move from staging to a separate production service and database.

A full chapter is devoted to collaboration with Members 1, 2, 4 and 5. In total Member 3 logged 194 hours over 67 working days. The report separates delivered, simulated and planned features: the payment path was built and tested in the provider's test mode, and live payments remain planned.

Keywords: online contest platform, payments, ledger, webhook, Razorpay, DevOps, security, load testing, Java, MySQL.

CONTENTS

1. INTRODUCTION....1
1.1 About the HackAgon Project....1
1.2 Purpose of This Report....1
1.3 Member 3 at a Glance....1
1.4 Sources and Method....1
1.5 Organisation of the Report....2
2. ROLE AND RESPONSIBILITIES....2
2.1 The Two Modules....2
2.2 Files, Tables and Endpoints....2
2.3 Decisions Owned by Member 3....3
2.4 What Belonged to Other Members....3
2.5 Effort Summary....3
3. REQUIREMENTS AND DESIGN....5
3.1 Requirements Covered....5
3.2 Non-Functional Requirements That Shaped the Work....5
3.3 Position in the Architecture....5
3.4 Data Design....6
3.5 Design Principles of the Money Module....6
4. MODULE 5: TOURNAMENTS AND MONEY....7
4.1 Money Tables and Ledger (Week 5)....7
4.2 Razorpay Test Account and the Money API Contract....7
4.3 Host Funding (Week 6)....7
4.4 Entry Fees (Week 7)....7
4.5 Refunds....8
4.6 Payout Details (Week 8)....8
4.7 The Payout Job: 1% Fee, Rank Split and Leftover Paise....8
4.8 Ledger Balance Check....8
4.9 Preparing for Real Money (September)....9
4.10 The First Real Tournament....9
5. MODULE 6: PLATFORM CORE AND DEVOPS....9
5.1 Audit and the Database Decision (Week 1)....9
5.2 Schema V1, Migrations and the Repository Layer (Week 2)....9
5.3 Smoke Checks and Staging Deployment....10
5.4 Security Middleware (Week 3)....10
5.5 Scoreboard Queries, Indexes, Backups and Milestone 1 (Week 4)....10
5.6 Security Review (Week 9)....10
5.7 Load Simulation and Tuning (Week 10)....10
5.8 Email Provider....11
5.9 Production Set-Up (Week 11)....11
5.10 Launch Week and After....11
6. COLLABORATION WITH OTHER MEMBERS....11
6.1 Overview of Dependencies and Joint Functions....12
6.2 Collaboration with Member 1 (Hiten Sharma)....13
6.3 Collaboration with Member 2 (Harsh Sharma)....14
6.4 Collaboration with Member 4 (Arjun Suman)....14
6.5 Collaboration with Member 5 (Akshat Rathore)....15
6.6 Team-Wide Activities....15
6.7 Summary of Collaboration....15
7. TESTING AND VERIFICATION....16

7.1 Test Strategy....16
7.2 Tests Recorded in the Work Log....16
7.3 Results from the Main Report's Smoke Test....16
7.4 Problems Found and Fixed....16
7.5 Known Issues That Remain....17
8. WEEK-BY-WEEK SUMMARY....17
8.1 Weeks 1 and 2: Audit and Foundation....17
8.2 Weeks 3 and 4: Protection and the First Playable Match....18
8.3 Weeks 5 and 6: Money Tables and Funding....18
8.4 Weeks 7 and 8: Fees, Refunds and Payouts....18
8.5 Weeks 9 and 10: Security and Load....18
8.6 Weeks 11 and 12: Production and Launch....18
8.7 Final Period: Closing the Loop....18
9. CHALLENGES, DECISIONS AND LIMITATIONS....18
9.1 Challenges and How They Were Met....18
9.2 Delivered, Simulated and Planned....19
9.3 Notes on the Sources....19
9.4 Limitations....20
10. CONCLUSION AND FUTURE SCOPE....20
10.1 Conclusion....20
10.2 Lessons Learnt....20
10.3 Future Scope....20
REFERENCES....21
APPENDIX A: COMPLETE DAILY WORK LOG OF MEMBER 3....21
APPENDIX B: DATED JOINT WORK WITH OTHER MEMBERS....23

LIST OF FIGURES

Figure 2.1: Hours logged by Member 3 per area of work.....4
Figure 2.2: Timeline of Member 3's work across the project weeks.....4
Figure 3.1: Life of a paid tournament in the money module.....6
Figure 6.1: Collaboration map of Member 3 with the other four members.....13

LIST OF TABLES

Table 1.1: Summary of Member 3.....1
Table 2.1: Files and responsibilities of Member 3.....2
Table 2.2: Hours by area of work.....3
Table 3.1: Requirements and Member 3's contribution.....5
Table 3.2: Money tables added by Member 3.....6
Table 4.1: Host funding work (weeks 5 and 6).....7
Table 4.2: The 1% fee on a sample pool (arithmetic only).....8
Table 4.3: Rules of the money module.....8
Table 5.1: Protective layer built in Week 3.....10
Table 5.2: Security review of 31 August to 4 September.....10
Table 5.3: Deployment stages.....11
Table 6.1: Dependencies of Member 3's modules (from the plan and logs).....12
Table 6.2: Matrix of joint functions.....12
Table 7.1: Tests carried out by Member 3.....16
Table 8.1: Weekly summary of Member 3's work.....17
Table 9.1: Challenges and responses.....18
Table 9.2: Status of Member 3's features.....19
Table A.1: Daily work log of Member 3.....21
Table B.1: Dated joint work.....23

1. INTRODUCTION

1.1 About the HackAgon Project

HackAgon is a web platform for hosting and joining online coding competitions. A college club, a class or a group of friends can set up a contest in a few minutes, choose questions from a shared bank, invite players and, when they want real stakes, run a paid tournament in which the host funds the prize pool in advance and the platform pays the winners by a published split after a flat 1% fee. The system uses Java and Spring Boot on the server, a MySQL-compatible database on TiDB Cloud, and fifteen plain HTML pages in the browser. According to the main project report it exposes 76 REST endpoints across eight controller classes and uses a twenty-table schema.

Five students of Arya College of Engineering and IT, Jaipur, built it between 6 July 2026 and 6 October 2026. The work was cut into ten modules and every member carried two of them. This report belongs to **Member 3, Hemanth Singh Tanwar**, whose area in the team plan is **Money, Core and DevOps**.

1.2 Purpose of This Report

The main project report describes the whole system and gives each module a short paragraph. It has no room for the full story of one person. This individual report fills that gap for Member 3: what was built, in which week, how it was checked, and how it fitted with the work of the other four members.

Money and platform plumbing are the two parts of HackAgon that every other part leans on. A player cannot join a paid room unless the entry-fee code accepts the payment; a result cannot be paid unless the rank list, the dispute check and the payout job agree; and none of it runs unless the schema, the repository layer and the deployment are sound. For this reason the report pays special attention to the places where Member 3's work meets that of the others, and Chapter 6 is devoted to them.

1.3 Member 3 at a Glance

Table 1.1: Summary of Member 3

Item	Details
Name	Hemanth Singh Tanwar (Member 3)
Role in the team plan	Money, Core and DevOps
Modules owned	Module 5: Tournaments and Money (weight 3) Module 6: Platform Core and DevOps (weight 2)
Main files	MoneyController.java, DatabaseService.java, application.properties, schema.sql, the migration runner, the seed script, the smoke-test script, RepoService.java, IdUtil.java, HackAgonApplication.java
Main tables	tournaments, prize_splits, payments, wallet_ledger; schema ownership of the whole database
External services	Razorpay (test account), Render (staging and production), TiDB Cloud (database), transactional email provider
Project period	6 July 2026 to 6 October 2026
Work logged	67 weekdays, all marked Completed; 194 hours (2 to 4 hours a day, 14.5 in each full week)
Collaborated with	Member 1 (Hiten Sharma), Member 2 (Harsh Sharma), Member 4 (Arjun Suman), Member 5 (Akshat Rathore)

1.4 Sources and Method

The report was written from three documents: the HackAgon Daily Work Log, the Team Modules and 12-Week Plan, and the facts about the delivered system that the team's main project report records. The

log gives the day-by-day record, including the 67 entries of Member 3 and the entries of the others that touch Member 3's work. The plan gives the module split, the weights, the weekly tasks, the milestones and the risk table. Dates follow the day-month-year style of the log.

Where the sources differ, one rule is used. The main project report is the authority on what the delivered system contains and which parts are real or simulated; the work log is the authority on what Member 3 did and on which day. Section 9.3 lists the places where the sources do not line up, so that the reader can judge them.

1.5 Organisation of the Report

Chapter 2 sets out the role, the files and the effort. Chapter 3 gives the requirements and the design. Chapter 4 covers Module 5, the money side, and Chapter 5 covers Module 6, the platform core and DevOps. Chapter 6 deals with collaboration. Chapter 7 reports testing, Chapter 8 follows the project week by week, and Chapter 9 discusses challenges, decisions and limits. Chapter 10 concludes. Appendix A reproduces the complete daily log of Member 3 and Appendix B lists the dated joint work with the other members.

2. ROLE AND RESPONSIBILITIES

2.1 The Two Modules

The plan gives every member one heavier and one lighter module so that the load stays equal; each member carries five weight points. Member 3 holds Module 5 (weight 3) and Module 6 (weight 2). Between them they decide whether money moves correctly and whether the platform stays up, safe and recoverable.

Module 5: Tournaments and Money

This module covers the paid side of HackAgon: creating a tournament and its prize split, funding the prize pool, charging entry fees, refunding when a tournament is cancelled or does not reach its minimum number of players, collecting payout details, and paying winners after a 1% platform fee. Everything that changes a balance is written to an append-only ledger. The plan also lists the rules page and the checks on payout accounts under this module.

Module 6: Platform Core and DevOps

This module is the ground the other nine modules stand on. It holds the database schema, the migration runner, the repository layer, the configuration, the security middleware, the smoke tests, the deployment on Render and the backups. It is also the module that carries the shared files: the plan states that schema.sql, HackAgonApplication.java and api.js are changed only through Member 3 (database and core) and Member 5 (browser scripts), and that everyone else asks by pull request.

2.2 Files, Tables and Endpoints

Table 2.1: Files and responsibilities of Member 3

File	Module	Responsibility
MoneyController.java	5	Tournament, funding, entry-fee, refund and payout routes; the only route file that moves money
create-tournament, tournaments and tournament-details pages	5	Browser pages of the paid flow; the list, rules, checkout, payout-status and winner screens were designed and built with Member 5
schema.sql and the migration runner	6	Table definitions (schema V1 fixed on 13 July) and the runner that applies them; shared file, changed through Member 3
RepoService.java	6	Repository layer: named functions for database access, including the scoreboard queries

File	Module	Responsibility
DatabaseService.java and application.properties	6	Connection to the database and the configuration read from the environment
HackAgonApplication.java and IdUtil.java	6	Server entry point and shared helpers; shared files
The seed script	6	Seed script; cleaned on 9 July so that it loads reference data only
The smoke-test script	6	Automatic checks of the running system, started on 16 July
The make-admin script and the fix-admin script	6	Helper scripts owned by others, locked by Member 3 in production on 4 September

Table 2.1 lists files by responsibility. The numbers of endpoints per route file are given in the main report and are not repeated here, because the work log does not break them down. What the log does show clearly is the sequence in which the money routes were added, and Chapter 4 follows that sequence.

2.3 Decisions Owned by Member 3

- **The ledger is append-only.** A balance is never edited. A new row is added for every movement, so any total can be rebuilt and checked afterwards.
- **Registration waits for the pool.** A paid tournament stays closed to players until the host's prize pool has been paid and confirmed by the payment provider.
- **Events are idempotent.** A repeated message from the payment provider must not credit or pay twice, so the webhook ignores events it has already handled.
- **Money is counted in paise.** Prize splitting is done on whole paise, with the leftover handled explicitly, instead of with fractions of a rupee.
- **Payout only after confirmation.** The payout job works from the confirmed rank list sent by the room module and does not run while a dispute is open.
- **Dangerous scripts are locked in production.** The make-admin script and the fix-admin script cannot be reached once the system is deployed.
- **Production is separate.** The live service and the live database are different from staging, with backups and a tested restore.

2.4 What Belonged to Other Members

Member 3 did not own the accounts and OTP code or the room and match logic (Member 1), the judge, the question bank and the admin screens (Member 2), the proctoring and duel code (Member 4), or the look of the screens, the learning pages and the AI hints (Member 5). In each of these areas Member 3 supplied a foundation or consumed a piece with a clear interface; Chapter 6 describes them.

2.5 Effort Summary

Member 3 logged work on all 67 weekdays from 6 July to 6 October 2026, a total of 194 hours. Daily entries run from 2 to 4 hours. Each of the twelve full weeks holds 14.5 hours (174 hours in all), and the final period from 28 September to 6 October holds seven days and 20 hours. The entries were grouped into eight areas of work.

Table 2.2: Hours by area of work

Area of work	Days	Hours	Share
Database and repository layer	8	23.5	12.1%
Security and middleware	10	31.0	16.0%

Area of work	Days	Hours	Share
Deployment, backups and production	13	35.0	18.0%
Testing and load	7	19.5	10.1%
Money setup and host funding	10	29.0	14.9%
Entry fees, refunds and payouts	10	29.0	14.9%
Launch support and compliance	7	21.5	11.1%
Review and documentation	2	5.5	2.8%
Total	67	194.0	100%

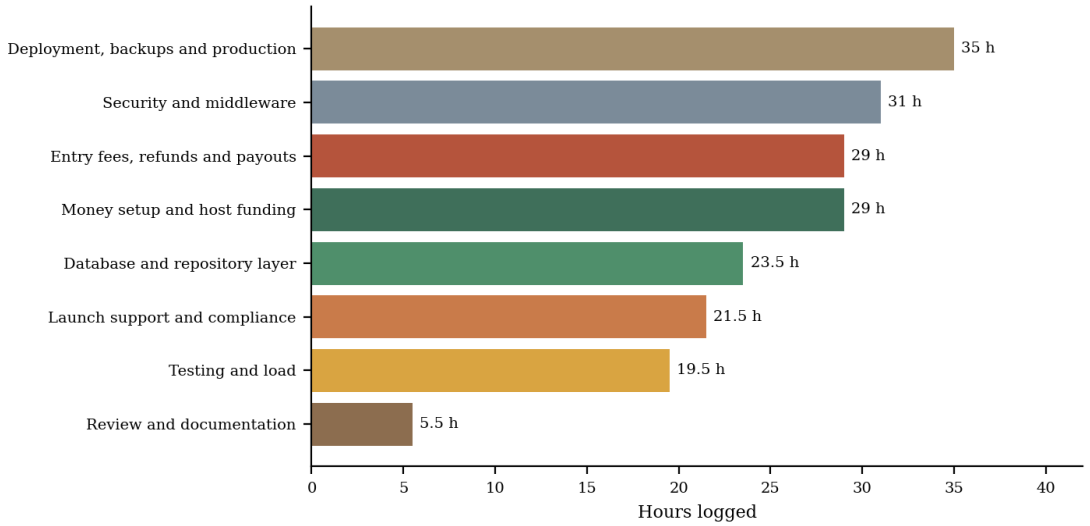


Figure 2.1: Hours logged by Member 3 per area of work

The biggest share, 35 hours, went into deployment, backups and production, because the project ran on staging for most of its life and moved to a separate production service only in September. Security and middleware came second, in two blocks: the protective layer built in Week 3 and the security review of Week 9. The two money areas together took 58 hours. Figure 2.2 shows when each area was active.

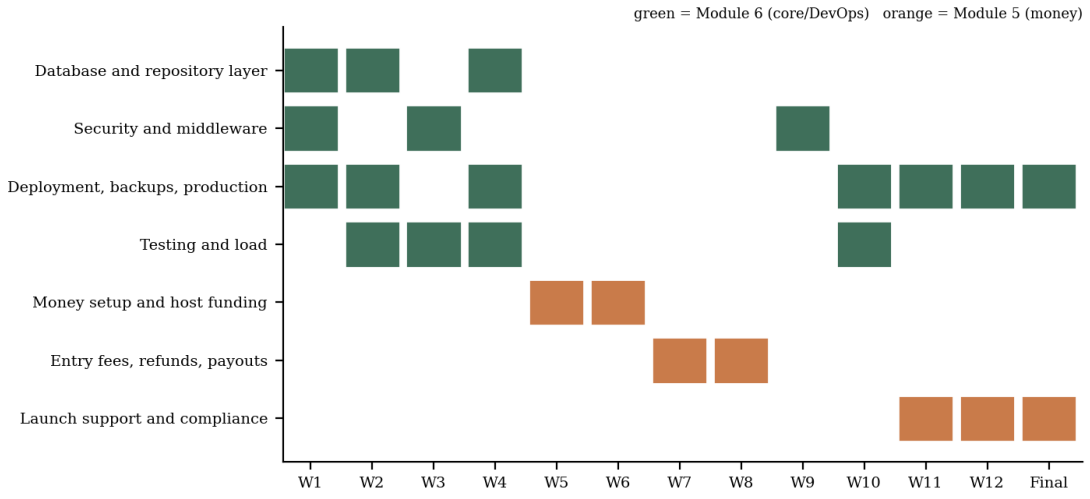


Figure 2.2: Timeline of Member 3's work across the project weeks

3. REQUIREMENTS AND DESIGN

3.1 Requirements Covered

The main report lists functional requirements FR1 to FR8. The log and the plan place Member 3 as the main owner of FR6 and as a contributor to FR8; the other requirements are shared with the members named below.

Table 3.1: Requirements and Member 3's contribution

ID	Requirement (short form)	Contribution of Member 3
FR6	Finalise a room, settle the prize pool, keep a ledger	Money tables and ledger, host funding, entry fees, refunds, payout details, payout job with the 1% fee, split by rank, ledger balance check
FR8	Administrator tools including money totals	Ledger and totals that the admin money tab reads; payout records that the failed-payout screen of Member 2 lists
FR4, FR5	Judging and proctoring (Members 2 and 4)	Schema, tables and storage on which their data lives; scoreboard queries and indexes; protection of every route
Non-functional	Security, reliability, deployability	Rate limits, validation, CORS, one error handler, parameterised queries, production separation, backups with a tested restore

3.2 Non-Functional Requirements That Shaped the Work

- **Correctness of money.** No rupee may be created or lost. Every movement is recorded, and the ledger must balance after a payout.
- **Security.** Every route uses parameterised queries; login and OTP are rate limited; secrets stay out of the repository; admin helper scripts are locked in production.
- **Reliability under load.** A room with hundreds of players polling the scoreboard must stay responsive, so queries are indexed and the scoreboard has a short cache.
- **Recoverability.** Backups are on, restore has been tested, and error alerts are set up.
- **Maintainability.** One repository layer, one error handler, migrations instead of hand-edited tables, and a single contract file for each pair of modules.

3.3 Position in the Architecture

HackAgon is a three-tier application served from one origin. The browser calls the JSON API under /api; the server talks to the database through a pool of connections and to outside services for payments, email and AI hints. Member 3's two modules sit on the server tier and around it: Module 6 is the database layer, the middleware and the hosting, and Module 5 is the one controller class that talks to the payment provider.

Figure 3.1 shows the life of a paid tournament as it passes through the money module. The top row is the collection side: the host funds the pool, registration opens, and players pay. The bottom row is the settlement side: the result is confirmed, a dispute check can hold the payout, the payout job splits the prize, and the ledger is checked.

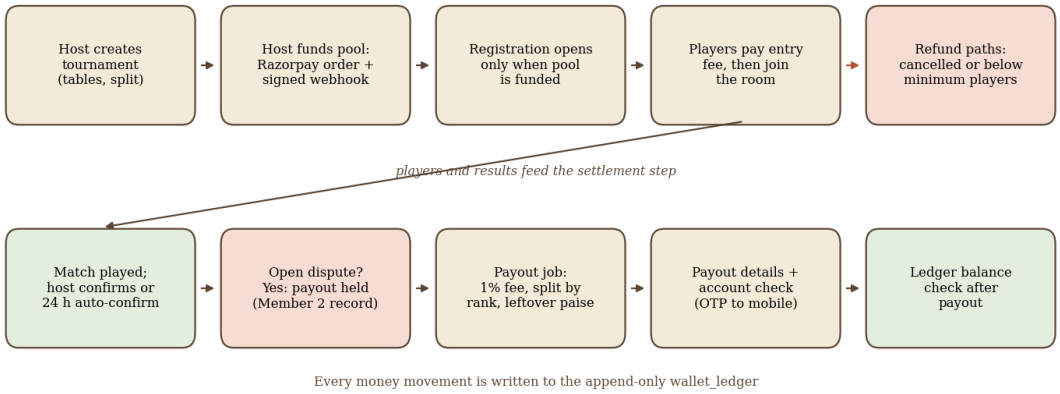


Figure 3.1: Life of a paid tournament in the money module

3.4 Data Design

Module 6 owns the schema of the whole database. The schema is applied by the migration runner, so a new environment can be built from nothing by running the migrations. Member 3 fixed the V1 tables on 13 July, finished the runner on 14 July and the repository layer on 15 July. The four tables of Module 5 were added in Week 5.

Table 3.2: Money tables added by Member 3

Table	Purpose	Created
tournaments	The paid tournament attached to a room: its entry fee, pool and status	03-08-2026
prize_splits	The share of the prize pool that goes to each rank	03-08-2026
payments	Payments made through the payment provider, such as pool funding and entry fees	04-08-2026
wallet_ledger	Append-only record of every money movement; the source for balances and totals	05-08-2026

Column-level detail is in the main report. The log records the order of creation and the rule that the ledger is append-only, and this report keeps to those facts.

3.5 Design Principles of the Money Module

- **Record first, then trust.** A balance is a sum of ledger rows, never a stored number that someone can overwrite.
- **Verify before acting.** The webhook checks the signature of each message before it changes anything.
- **Do things once.** Repeated provider events are ignored, so retries by the provider are harmless.
- **Close the door until it is safe.** Registration stays closed until the pool is funded, and a payout waits for a confirmed result and no open dispute.
- **Keep the screens thin.** The failed-payout and retry screens of Member 2 only show and trigger; the logic stays in the money module.

4. MODULE 5: TOURNAMENTS AND MONEY

This chapter follows the order in which the money module was built: tables and contract in Week 5, host funding in Week 6, entry fees and refunds in Week 7, payouts in Week 8, and then the preparation for real use in September.

4.1 Money Tables and Ledger (Week 5)

The money work began on 3 August with the tournaments and prize_splits tables, followed by the payments table on 4 August. On 5 August, the longest day of that week at four hours, Member 3 created the append-only wallet_ledger. The ledger is the core of the module: when funding, entry fees, refunds, fees and payouts all write to the same ledger, the question "where did the money go?" can be answered by reading rows.

4.2 Razorpay Test Account and the Money API Contract

On 6 August Member 3 opened a Razorpay test account, and on 7 August wrote the money API contract together with Member 1 and Member 5. The contract lists the requests and replies that the browser pages and the accounts module use, so that Member 5 could design the money screens, and Member 1 could send the rank list, without waiting for the routes to be finished. This follows the plan's rule of contracts first.

4.3 Host Funding (Week 6)

Table 4.1: Host funding work (weeks 5 and 6)

Date	Work	Purpose
03-08-2026	Tournaments and prize_splits tables	Store the tournament and its split
04-08-2026	Payments table	Store payments from the provider
05-08-2026	Append-only wallet_ledger	Record every movement
06-08-2026	Razorpay test account	Safe place to try the gateway
07-08-2026	Money API contract with Members 1 and 5	Agree requests and replies early
10-08-2026	Razorpay order for the prize pool	Let the host pay the pool
11-08-2026	Webhook with signature check	Accept only genuine provider messages
12-08-2026	Webhook ignores repeated events	Make retries harmless
13-08-2026	Registration closed until pool is funded	No players join an unfunded prize
14-08-2026	Host funding tested in test mode	Prove the whole path once

The funding flow has three parts. First, the server creates a Razorpay order for the prize pool. Second, the provider tells the server about the payment through a webhook. The webhook checks the signature, so a forged message is refused, and it ignores an event it has seen before, because payment providers may deliver the same event more than once. Third, only when the pool is confirmed as funded does the tournament open for registration. The last step was a decision about trust: a player should never pay to enter a contest whose prize does not exist yet.

4.4 Entry Fees (Week 7)

On 17 August Member 3 built the entry-fee checkout, and on 18 August made the server join the player to the room after the payment is confirmed. The order matters. The player is added to the room as a result of a confirmed payment and not of a click in the browser. The room-joining logic itself belongs to Member 1, who built join by Arena and invite code on 27 July; the money module calls into it after

payment. The same week Member 1 added the rule that only users aged 18 or over may enter or host paid tournaments, and Member 1 tested elimination and the age rule on 20 August.

4.5 Refunds

Two refund paths were built. On 19 August refunds were added for cancelled tournaments, and on 20 August for tournaments that did not reach the minimum number of players. In both cases the money goes back through the same ledger, so a refund is simply a new row that cancels the effect of an earlier one. On 21 August Member 3 prepared the data for the rules page, so that the page shown to players states the fee, the split and the refund conditions from real tournament records and not from text typed into the page.

4.6 Payout Details (Week 8)

A winner has to say where the money should go. On 24 August Member 3 built the payout details feature, which sends a one-time password to the mobile number on the player's profile before details can be saved. On 25 August the Razorpay account validation was added, so that an invalid account is caught when the details are entered and not when the payout fails. The OTP step reuses the SMS OTP that Member 1 built on 15 July; the profile mobile is the number Member 1's profile edit protects.

4.7 The Payout Job: 1% Fee, Rank Split and Leftover Paise

The payout job was built on 26 August and the prize split on 27 August. The job takes the final rank list that the room module sends once results are confirmed, deducts the 1% platform fee, divides what remains by the prize split, and pays each ranked winner. Dividing money by percentages in whole paise usually leaves a few paise over, and on 27 August Member 3 added explicit handling for that leftover, so that the total paid out plus the fee always equals the pool and nothing disappears into rounding.

Table 4.2: The 1% fee on a sample pool (arithmetic only)

Step	Amount
Prize pool funded by the host	Rs 5,000.00
Platform fee at 1%	Rs 50.00
Amount left to split by rank	Rs 4,950.00

The figures in Table 4.2 only show how the 1% fee is calculated. They are an example, not test data from the project, and the real split between ranks comes from the prize_splits table.

4.8 Ledger Balance Check

The week ended on 28 August with a ledger balance check. After money has moved, the check adds up the ledger and confirms that the movements agree with each other. This is the safety net behind the rule that no money is created or lost. In the same week Member 1 and Member 3 tested result locking together (27 August) and Member 5 built and tested the money screens (24 to 28 August), so by the end of Week 8 the full paid path had been seen from three sides.

Table 4.3: Rules of the money module

Rule	How it is enforced	Built
Only genuine provider messages count	Webhook checks the signature	11-08-2026
A provider event is applied once	Webhook ignores repeated events	12-08-2026
No entry before the prize exists	Registration closed until the pool is funded	13-08-2026
A player pays before joining	Join after payment is confirmed	18-08-2026

Rule	How it is enforced	Built
Money returns when the contest fails	Refunds for cancelled and under-minimum tournaments	19 to 20-08-2026
Payout goes to a verified account	OTP to profile mobile and account validation	24 to 25-08-2026
Nothing is lost in rounding	Leftover paise handled in the split	27-08-2026
Books always balance	Ledger balance check	28-08-2026

4.9 Preparing for Real Money (September)

On 16 September Member 3 wrote the Razorpay live-mode checklist, and on 17 September filed the written confirmation about paid contests. The plan names the reason: holding player money needs the payment partner's approval, and the fallback is to keep paid mode behind a switch and launch free tournaments first if needed. These two entries are the paper side of the money module, and they sit on the same days as Member 1's refund policy (16 September) and webcam consent text (17 September), so the legal pages and the money rules were written against each other.

4.10 The First Real Tournament

On 23 September Member 3 ran one small real tournament with friends, the day after deploying to production. On 24 September Member 3 checked every payout, and on 25 September checked the ledger and joined the team retro. The checking continued after the event: the ledger again on 28 September, and the Razorpay webhook logs on 29 September. This is the plan's Milestone 3 test: production is live and one real tournament has finished with correct payouts. The log records that every payout was checked; whether real money or the simulated gateway moved in this build is stated in the main report, and Section 9.3 explains how this report treats that point.

5. MODULE 6: PLATFORM CORE AND DEVOPS

Module 6 has less to show on screen than Module 5, but it was needed from the first day. This chapter follows the same order as the work: the audit, the foundation, the middleware, the first deployment, the security review, the load tests and the move to production.

5.1 Audit and the Database Decision (Week 1)

The first week was audit. On 6 July Member 3 compared a MySQL host with PostgreSQL, the decision the plan asks the team to take first, because the code uses the MySQL Connector/J (JDBC) driver while Render's own database is PostgreSQL. The delivered system uses a MySQL-compatible database on TiDB Cloud, so the code did not have to change its driver. On 7 July the existing schema.sql was reviewed against the PRD tables. On 8 July, the four-hour day of the week, Member 3 checked that the secrets file and the target build folder are in .gitignore, so that secrets and build output never reach the repository. On 9 July fake data was removed from the seed files, so that every counter in the app starts from zero as the PRD requires. On 10 July a Render staging service with a health check was created.

5.2 Schema V1, Migrations and the Repository Layer (Week 2)

The foundation was completed in three days. On 13 July the schema V1 tables were fixed, on 14 July the migration runner was finished, and on 15 July the repository layer in RepoService.java was finished. The repository layer gives the controller classes named functions for database access, so that a route does not carry its own SQL scattered about. Fixing V1 early gave the others a stable base: Member 4 added the duel and team tables on 17 July, Member 5's mistakes list and achievements table followed in August, and Member 2's questions and submissions tables sit in the same schema.

5.3 Smoke Checks and Staging Deployment

On 16 July Member 3 added the first smoke-test checks, and on 17 July set up automatic deployment to staging from the main branch. From that day every merge to main produced a running copy that the whole team could try. This supports the plan's rule that main is always deployable and that each Friday shows what works in the browser.

5.4 Security Middleware (Week 3)

Table 5.1: Protective layer built in Week 3

Date	Work	Protects against
20-07-2026	Rate limits on login and OTP	Guessing passwords and OTP codes, and flooding SMS and email
21-07-2026	Input validation	Bad or hostile data reaching the database
22-07-2026	CORS rules	Pages from other origins calling the API
23-07-2026	One error handler and request logs	Unhandled crashes and untraceable requests
24-07-2026	Tests for auth routes	Regressions in login and OTP

The layer was placed in front of every route, so it protects the work of all members equally. It also fits the work of Member 1: the OTP and the 5-attempt and 60-second resend rules were built on 13 and 14 July, and the rate limit of 20 July sits in front of them.

5.5 Scoreboard Queries, Indexes, Backups and Milestone 1 (Week 4)

In Week 4 the match became playable. On 27 July Member 3 added repository functions for the scoreboard queries and on 28 July the first database indexes. On 29 July backups were turned on, the day of the week with four hours. On 30 July Milestone 1 was deployed to staging, and on 31 July the test checklist was run and the issues found were fixed. The first playable match (two users playing a full solo room, with a new account showing zero everywhere) was ready for the milestone on 2 August.

5.6 Security Review (Week 9)

Table 5.2: Security review of 31 August to 4 September

Date	Activity	Result recorded
31-08-2026	Checked that every route uses parameterised queries	Routes reviewed
01-09-2026	Tested SQL injection	Tests run
02-09-2026	Tested session fixation and token misuse	Tests run
03-09-2026	Fixed security findings	Findings fixed
04-09-2026	Locked the make-admin script and the fix-admin script in production	Scripts unreachable in production

The review covered all controller classes, not only Member 3's, and the findings were fixed on 3 September. The last item is the counterpart of Member 2's work of 7 August, when the make-admin script was protected and the fix-admin script removed from deployment: one member closed the risk in the code, the other in the deployed system. The log does not list the individual findings, so none are claimed here.

5.7 Load Simulation and Tuning (Week 10)

On 7 September Member 3 wrote a load simulation for 200 players polling one room, and on 8 September ran it up to 500 players. The findings were used at once: indexes were added on 9 September,

and a short cache for scoreboards on 10 September. The cache matters because a scoreboard is read far more often than it changes, so even a very short cache saves most of the database work. The log records what was done with the results and not the response times, so this report does not quote figures. In the same days Member 2 sent 200 queued submissions to the judge, so the team tested two different doors of the same server.

5.8 Email Provider

On 11 September email was moved from Gmail to a transactional provider. Gmail is convenient for development but has sending limits and is not made for sending OTP codes to many users. The change affects Member 1's OTP and password-reset mail, which depend on OtpService.java (mail delivery).

5.9 Production Set-Up (Week 11)

Table 5.3: Deployment stages

Date	Work	Purpose
10-07-2026	Render staging service with health check	First running copy
17-07-2026	Automatic deploy to staging from main	Every merge produces a test copy
29-07-2026	Backups on	Protect data from the first milestone
30-07-2026	Milestone 1 deployed to staging	First playable match
14-09-2026	Separate production service	Keep live traffic away from testing
15-09-2026	Separate production database	Keep live data away from test data
18-09-2026	Backup restore test; error alerts	Prove recovery; learn of faults early
21-09-2026	Final pre-deploy checks	Last gate before launch
22-09-2026	Deployed to production	Go live
30-09-2026	Deploy and backup issues fixed	Close what the first event showed
01-10-2026	Backups verified to restore	Confirm recovery again after launch
02-10-2026	Deployment notes updated	Leave a record for the next maintainer

The plan's risk table says free hosting limits can put the project at risk: free services sleep, local files are lost and databases may expire. The answer was to separate production from staging in both service and database (14 and 15 September), to test restore (18 September) and to set error alerts.

5.10 Launch Week and After

Member 3 completed the final pre-deploy checks on 21 September, the same day that Member 1 recorded the code freeze, and deployed to production on 22 September. The week after the launch was spent on closing small gaps: deploy and backup issues found during the first event were fixed on 30 September, backups were verified again on 1 October, and the deployment notes were updated on 2 October. On 5 October Member 3 reviewed the pull requests of teammates, and on 6 October took part in the final review and submission preparation.

6. COLLABORATION WITH OTHER MEMBERS

No HackAgon module runs alone, and Member 3's two modules touch every other. The plan states the pairs that depend on one another and says that the owners agree an interface before they build against it. This chapter takes the point of view of Member 3 and shows, for each other member, the shared functions, what each side built and the dates in the logs.

6.1 Overview of Dependencies and Joint Functions

Table 6.1: Dependencies of Member 3's modules (from the plan and logs)

Module	Depends on	Interface agreed with
Tournaments and money	Accounts and OTP (payout details), rooms (rank list and join), design system (screens)	Hiten Sharma, Akshat Rathore
Tournaments and money	Admin tools for disputes and failed payouts	Harsh Sharma
Platform core and DevOps	Every module: schema, shared files, deployment	Each owner, by pull request
Platform core and DevOps	Duel and team tables, proctor storage	Arjun Suman

Table 6.2: Matrix of joint functions

Joint function	With	Member 3 built	Other member built
Data model and schema	M1, M4	Schema V1, migration runner, repository layer	Room and match model draft (M1); duel and team tables (M4)
Login and OTP protection	M1	Rate limits, validation, auth tests, security review	Email and SMS OTP, forgot password, profile
Scoreboard data	M1, M5	Scoreboard queries, indexes, cache	Ranks and tie-break (M1); polling scoreboard (M5)
Money API contract	M1, M5	Contract and routes	Requests and replies used by accounts and screens
Entry fee and join	M1	Checkout, join after payment	Join logic, 18+ rule
Payout details	M1	Payout details with OTP, account validation	SMS OTP, profile mobile
Results to payout	M1, M2	Payout job, rank split	Confirm and auto-confirm, rank list (M1); dispute hold (M2)
Failed payouts	M2	Payout job and its failures	Failed-payout screen and retry button
Admin script security	M2	Lock in production	Protect the make-admin script, remove the fix-admin script
Money screens and rules page	M5	Rules-page data, checkout and payout routes	Tournament list, rules, checkout, payout status, winner screen
Load and end-to-end	M1, M2	Load simulation to 500 players	End-to-end script (M1); judge load test (M2)

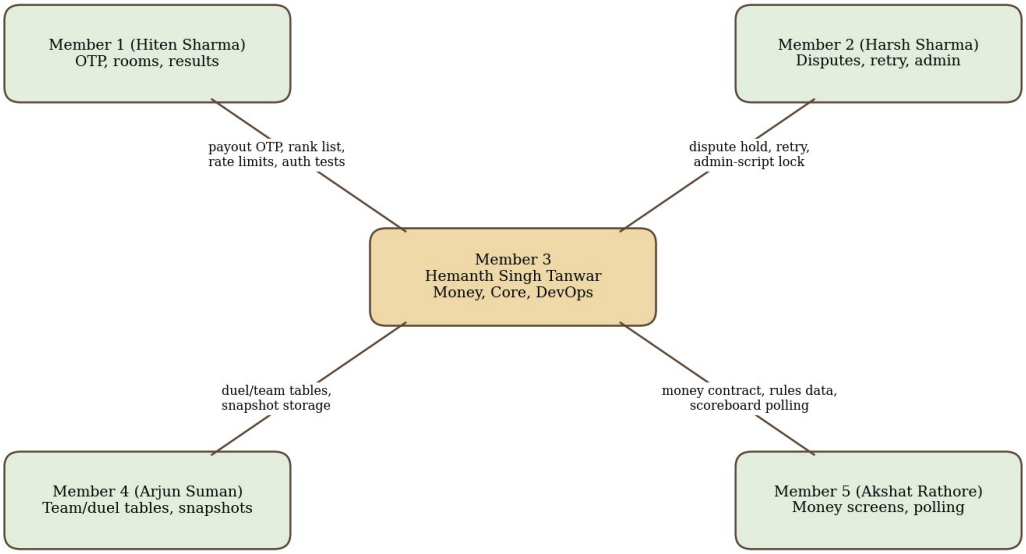


Figure 6.1: Collaboration map of Member 3 with the other four members

6.2 Collaboration with Member 1 (Hiten Sharma)

A. Data model and schema

On 9 July Member 1 drafted the room and match data model and on 10 July reviewed it with Member 3, who had been checking the schema against the PRD since 7 July. Member 3 then fixed the V1 tables on 13 July. The room and match tables of Member 1 therefore stand on a schema that both had looked at before it was frozen.

B. Protection of login and OTP

Member 1 built email OTP with a 5-minute expiry on 13 July, added the 5-attempt limit and 60-second resend on 14 July, built SMS OTP through MSG91 on 15 July, the forgot-password flow on 16 July and the profile edit on 17 July. Member 3 added the rate limits on login and OTP on 20 July and wrote tests for the auth routes on 24 July. In the security review of 31 August to 3 September Member 3 tested session fixation and token misuse, which concern the session handling that Member 1 had audited in the first week (6 to 8 July).

C. Entry fee, join and the age rule

The room join logic of Member 1 (27 July) and the entry-fee join of Member 3 (18 August) meet at one door. Member 1 added the 18+ rule for paid tournaments on 19 August and tested it, together with elimination, on 20 August. The money contract of 7 August was written with Member 1, so that the room module and the money module agreed on what is sent when a player pays.

D. Payout details and the rank list

Payout details use the SMS OTP and the profile mobile of Member 1. The result chain is also joint: Member 1 built final ranks and the tie-break on 3 August, the host confirm button on 24 August and the 24-hour auto-confirm on 25 August, and sent the final rank list to the money module on 26 August. Member 3 built the payout job on 26 August and the rank split on 27 August, and on 27 August the two tested result locking together. Because of this chain, a payout is never based on a result that the room module has not locked.

E. Organisation approval, tests and launch

Member 1's organisation approval flow (31 August to 2 September) ran at the same time as the security review, which covered it. Member 1's end-to-end script (7 to 9 September) and Member 3's load simulation (7 and 8 September) tested the same system from two sides, one for a complete path and the other for many players. Member 1's refund policy (16 September) was written beside Member 3's refund logic of 19 and 20 August. For the launch, Member 1 recorded the code freeze on 21 September, the day Member 3 did the final pre-deploy checks, and watched signups and OTP delivery from 22 to 24 September while Member 3 deployed and ran the first tournament. On 1 October Member 1 updated api-contract.md for accounts and rooms.

6.3 Collaboration with Member 2 (Harsh Sharma)

A. Disputes and payouts

Member 2 built the player dispute form on 17 August and made a dispute block the payout on 18 August. The money module reads the dispute record before it pays: an open dispute holds the payout until an administrator decides. The tool of Member 2 for deciding and closing disputes was built on 19 and 20 August, and tested on 21 August, before the payout job existed on 26 August. The hold was therefore ready and tested before the money module needed it.

B. Failed payouts and retry

A payout can fail for reasons outside the platform. Member 2 built the failed-payouts screen on 24 August and the retry button on 25 August. Member 3's payout job followed on 26 August. The division is clear: the screen presents and triggers, and the money module decides how money moves. On 28 August Member 2 tested the admin tools and Member 3 added the ledger balance check, so the screens and the ledger were tested in the same week.

C. Admin scripts and seed data

Member 2 protected the make-admin script and removed the fix-admin script from deployment on 7 August, and Member 3 locked both in production on 4 September. Member 3 had removed fake data from the seeds on 9 July, which allowed Member 2 to load only real starter questions on 22 and 23 July, and to confirm on 18 September that the seed files hold only reference data. The tables of Member 2 (questions, submissions, mistakes, books, disputes) are part of the schema that Member 3 fixed on 13 July.

D. Load, the first event and closing the loop

Member 2 sent 200 queued submissions to the judge on 7 and 8 September and tuned concurrency on 9 September, while Member 3 simulated 200 and then 500 players on the scoreboard and added indexes on the same day. After the first event Member 2 reviewed its disputes on 24 September, the day Member 3 checked every payout, and closed the open disputes on 29 September, the day Member 3 reviewed the webhook logs. The admin money tab, which shows the totals from the ledger, was one of the smoke-test checks reported in the main report.

6.4 Collaboration with Member 4 (Arjun Suman)

A. Tables and storage

Member 4 wrote the proctor API contract on 10 July, built the snapshot, violation and lock-state endpoints on 13 to 15 July, chose where snapshots are stored on 16 July and, on 17 July, added the duel and team tables to the schema with Member 3. Because the schema was fixed on 13 July and the runner finished on 14 July, these tables were added through the migrations and not by hand.

B. Backups and snapshot retention

Member 4 built the job that deletes snapshots after 30 days on 21 August and checked it again on 2 October. Member 3's backups, which started on 29 July, and the restore tests of 18 September and 1 October protect the database, whereas the plan's risk table warns that free hosting can lose local files and recommends keeping snapshots in external storage. The main report lists moving snapshots to object storage as future scope.

C. Test periods and the first event

Member 4's 10-person proctoring test and tuning (24 to 28 August) happened during the week in which the payout job was built, on the staging service that Member 3 maintained. Member 4's three mock matches (14 to 16 September) happened while Member 3 created the production service and database. During the first event Member 4 monitored locks on 22 September and snapshots on 23 September and fixed urgent problems on 24 September, while Member 3 deployed, ran the tournament and checked every payout. Member 4 reviewed the proctoring logs on 28 September, and Member 3 the ledger.

6.5 Collaboration with Member 5 (Akshat Rathore)

A. Money contract and money screens

The money contract was written with Member 5 on 7 August. The backend then led the screens by about a week: Member 3 prepared the rules-page data on 21 August, built payout details on 24 August, the payout job on 26 August; Member 5 built the tournaments list on 24 August, the rules page on 25 August, the checkout screen on 26 August, the payout status screen on 27 August and the winner screen, with a test of all money screens, on 28 August. Entry-fee checkout was ready on 17 August, nine days before its screen.

B. Scoreboard

Member 3 added the scoreboard queries on 27 July, and Member 5 built the polling scoreboard on 29 July for the match page. When the load simulation of 7 and 8 September showed how the scoreboard behaves with many players, Member 3 added indexes (9 September) and a short cache (10 September), which helped every screen that polls it.

C. Real numbers only

Member 3 removed fake data from the seed files on 9 July; Member 5 calculated the achievement metrics from real counts on 18 August (including tournaments won, which relies on tournament records) and removed invented numbers and testimonials from the landing page on 15 September. Together the two make sure that every counter in HackAgon starts from zero and grows only from real activity.

D. Feedback after the first event

Member 5 collected feedback from the first players on 24 September and reviewed it on 28 September. Member 3 checked the payouts the same day and the ledger four days later, so what players said and what the books showed could be compared.

6.6 Team-Wide Activities

In Week 1 every member audited or tested the existing code, and on 10 July each wrote the decisions that the others would build on; Member 3's was the creation of the staging service with its health check. The three shared milestones were the first playable match on 2 August, the full paid test tournament on 30 August and the first deployed release on 27 September. The whole team held the retro on 25 September. In the last week Member 3 reviewed teammates' pull requests on 5 October, and every member did the final review and submission preparation on 6 October. As owner of the shared files, Member 3 received change requests for schema.sql and HackAgonApplication.java from the others by pull request.

6.7 Summary of Collaboration

With Member 1 the shared object was the identity of the payer and the result that decides the payout; with Member 2 it was the hold and the retry around the payout; with Member 4 it was tables, storage and the testing of the system under stress; and with Member 5 it was the screens that show the money and the data behind them. In each case the interface was small and agreed before building, and the two sides were finished in neighbouring weeks, which is why a paid tournament could run from start to finish on 23 September.

7. TESTING AND VERIFICATION

7.1 Test Strategy

Four kinds of checking were used. **Automatic checks** (the smoke-test script and the tests for the auth routes) run against the running system. **Mode checks** try each money step in the provider's test mode before it is trusted. **Review and attack tests** went through every route for injection, session fixation and token misuse. **Stress and recovery tests** put hundreds of players on a room and restored backups into a clean place.

7.2 Tests Recorded in the Work Log

Table 7.1: Tests carried out by Member 3

Date	Test	Outcome or follow-up
16-07	The smoke-test script checks added	Automatic checks available from Week 2
24-07	Tests for auth routes	Login and OTP routes covered
31-07	Test checklist for Milestone 1	Issues found were fixed
14-08	Host funding in test mode	Funding path proved before entry fees were built
27-08	Result locking, with Member 1	Joint test of the confirm-to-payout chain
28-08	Ledger balance check	Check added and available for every payout
31-08 to 03-09	Security review	Findings fixed on 03-09
07-09, 08-09	Load simulation, 200 then 500 players	Indexes (09-09) and cache (10-09) added from findings
18-09	Backup restore test	Recovery path proved; alerts set up
24-09	Every payout of the first event checked	Done
25-09, 28-09	Ledger checked after the first tournament	Done
29-09	Razorpay webhook logs reviewed	Done
01-10	Backups verified to restore	Confirmed again after launch

7.3 Results from the Main Report's Smoke Test

The smoke-test script file is one of Member 3's files, and the main report describes the automatic smoke test that uses it. Of the cases reported there, the administrator money-totals check, which reads the ledger totals, passed. A run with a non-administrator account failed the administrator checks, which confirms that the gate works for the right reason. The checks that failed in the administrator run did not concern the money module: two were the AI-coach checks that need a key, and two were proctoring checks that failed because the script created a room without proctoring.

7.4 Problems Found and Fixed

After the first milestone the test checklist led to fixes on 31 July. The security review led to fixes on 3 September. The load simulation led to indexes and a cache. After the first real tournament, deploy and backup issues were found and fixed on 30 September. The log names the area but not each defect, so the report lists them only at this level.

7.5 Known Issues That Remain

- The payment gateway is simulated in the tested build according to the main report, so live-money behaviour has not been shown by this build.
- The load simulation outcomes are recorded as actions taken in the log and not as response times, so the report makes no performance claim.
- Browser, email and SMS delivery depend on outside providers whose limits are outside the project's control.
- Snapshots are stored with the application and are deleted after 30 days; moving them to object storage is future scope.

8. WEEK-BY-WEEK SUMMARY

This chapter follows the plan's twelve weeks and a final period. Every full week has five working days and 14.5 logged hours; the final period, from 28 September to 6 October, has seven days and 20 hours.

Table 8.1: Weekly summary of Member 3's work

Week	Dates	Focus	Hours
1	6 to 10 Jul	Database choice, schema review, .gitignore check, seed clean-up, Render staging	14.5
2	13 to 17 Jul	Schema V1, migration runner, repo layer, the smoke-test script, staging auto-deploy	14.5
3	20 to 24 Jul	Rate limits, validation, CORS, error handler, auth tests	14.5
4	27 to 31 Jul	Scoreboard queries, indexes, backups, Milestone 1 on staging, checklist	14.5
5	3 to 7 Aug	Tournaments, prize_splits, payments, wallet_ledger, Razorpay account, contract	14.5
6	10 to 14 Aug	Razorpay order, webhook, repeat-event rule, registration gate, funding test	14.5
7	17 to 21 Aug	Entry fee checkout, join after payment, two refund paths, rules-page data	14.5
8	24 to 28 Aug	Payout details with OTP, account validation, payout job, split, ledger check	14.5
9	31 Aug to 4 Sep	Security review, fixes, locking admin scripts in production	14.5
10	7 to 11 Sep	Load simulation, indexes, scoreboard cache, email provider	14.5
11	14 to 18 Sep	Production service and database, live-mode checklist, written confirmation, restore test	14.5
12	21 to 25 Sep	Pre-deploy checks, production deploy, first tournament, payout and ledger checks, retro	14.5
Final	28 Sep to 6 Oct	Ledger and webhook-log review, deploy and backup fixes, restore check, notes, review	20.0
Total			194.0

8.1 Weeks 1 and 2: Audit and Foundation

Week 1 was reading and deciding: the database host, the schema against the PRD, the ignore file, the seeds and a staging service. Week 2 built the base on which the rest depended: schema V1, the migration runner, the repository layer, the first smoke checks and automatic deployment to staging. Member 1 and Member 5 built accounts and components in the same weeks, so the data, the account and the interface foundations were laid together.

8.2 Weeks 3 and 4: Protection and the First Playable Match

Week 3 put protection in front of the whole server. Week 4 prepared data access, speed and recovery for the first match: scoreboard queries, indexes, backups and a staging deployment. Milestone 1, two users playing a full solo room, followed on 2 August.

8.3 Weeks 5 and 6: Money Tables and Funding

Week 5 created the four money tables and the Razorpay test account and wrote the contract with Members 1 and 5. Week 6 turned them into a working funding flow, with the three rules on signature, repeats and the closed registration.

8.4 Weeks 7 and 8: Fees, Refunds and Payouts

Week 7 let players pay and handled the cases where a tournament fails. Week 8 collected payout details, paid winners by the split after the 1% fee and checked the ledger. The week matched the busiest weeks of Members 1, 2 and 5 on results, disputes and money screens. Milestone 2, the full paid test tournament, followed on 30 August.

8.5 Weeks 9 and 10: Security and Load

Week 9 attacked the system's own routes, and fixed what was found. Week 10 sent hundreds of simulated players at one room, and used the findings for indexes and a cache. The switch from Gmail to a transactional email provider was made on 11 September.

8.6 Weeks 11 and 12: Production and Launch

Week 11 created production as a separate service and database, wrote the live-mode checklist, filed the written confirmation and tested restore. Week 12 held the final checks, deployed on 22 September and ran the first real tournament on 23 September. Milestone 3, the first deployed release, was on 27 September.

8.7 Final Period: Closing the Loop

In the final period Member 3 checked the ledger after the first tournament (28 September), reviewed the webhook logs (29 September), fixed the deploy and backup issues (30 September), verified that backups restore (1 October), updated the deployment notes (2 October), reviewed teammates' pull requests (5 October) and took part in the final review (6 October).

9. CHALLENGES, DECISIONS AND LIMITATIONS

9.1 Challenges and How They Were Met

Table 9.1: Challenges and responses

Challenge	Response of Member 3
Money must not be created or lost	Append-only ledger, one row per movement, and a ledger balance check after payouts

Challenge	Response of Member 3
The provider may send the same event twice	Webhook checks the signature and ignores repeated events
Players could pay into a prize that does not exist	Registration stays closed until the pool is funded
Splitting money leaves rounding leftovers	Whole-paise arithmetic with explicit handling of leftover paise
A tournament may fail after players paid	Two refund paths: cancelled, and under the minimum players
Payout details are sensitive	OTP to the profile mobile and account validation before saving
Dangerous helper scripts	Locked in production, as the counterpart of Member 2's protection in code
Many players polling one room	Load simulation to 500 players, then indexes and a short scoreboard cache
Free hosting can lose files or sleep	Separate production service and database, backups, tested restore, error alerts
Shared files touch every module	Changes only through Members 3 and 5; others by pull request
Dependence on four other members	Contract files and small interfaces agreed before building

9.2 Delivered, Simulated and Planned

The main report keeps real, simulated and planned features apart, and this report does the same for the scope of Member 3.

Table 9.2: Status of Member 3's features

Feature	Status in this build
Schema V1, migrations, repository layer, seed clean-up	Delivered
Rate limits, validation, CORS, error handler, request logs, auth tests	Delivered
Parameterised-query review, injection and session tests, fixes	Delivered
Staging deployment, production service and database, backups with restore test	Delivered
Money tables and append-only ledger	Delivered
Razorpay order, signed webhook, funding, entry fee, refunds, payout job	Delivered in test mode; the main report describes the gateway as simulated in the tested build
Payout details with OTP and account validation	Delivered
Live-mode payments	Planned: checklist written on 16-09 and written confirmation filed on 17-09; approval status is not recorded in the log
Transactional email	Moved on 11-09; see Section 9.3

9.3 Notes on the Sources

The log, the plan and the main report were written for different purposes, and some details do not line up. These points are given so the reader can judge them.

- The log of Member 3 refers to a Razorpay test account, a webhook with a signature check and a live-mode checklist, while the main report calls the payment gateway simulated in the tested build. This report follows the log for what was done and the main report for what the delivered system contains, and does not claim that real money moved.

- The plan lists create-tournament, tournaments and tournament-details pages among the files of Module 5, while the log shows that Member 5 built the tournaments list, the rules page, checkout, payout status and winner screens (24 to 28 August). Member 3's side was the data and the routes.
- The plan speaks of UPI validation, and the log of Razorpay account validation (25 August). This report uses the log's term.
- The log shows the move from Gmail to a transactional provider on 11 September, while the main report names Gmail SMTP among the outside services. The log is followed for the activity.
- The log of Member 3 places the first small real tournament on 23 September, while the logs of Members 1, 2 and 4 show monitoring and review on 22 to 24 September. This report treats the event as spanning 23 to 24 September, with payouts checked on 24 September.
- The log gives the name of each security check but not the findings, and the load simulation but not its figures; no findings or figures are quoted.

9.4 Limitations

- Live payments need approval from the payment partner; until then paid mode is only proved in test conditions.
- The money module relies on a single payment provider; there is no second route if it is unavailable.
- Reconciliation between the provider's records and the ledger is by manual review of logs (29 September) and not an automatic job.
- Backups and restore have been tested, but there is no recorded fail-over to a second region.
- The security review is a project-team review and not an outside audit.

10. CONCLUSION AND FUTURE SCOPE

10.1 Conclusion

Member 3 delivered the two modules that let HackAgon handle money and stay running. The money module turns a host's funding, players' entry fees, refunds and winners' payouts into rows of an append-only ledger, guards each step with a rule, and was run once with a real group on 23 September. The core module gave the project a schema, migrations, a repository layer, a protective layer in front of every route, a security review, a load simulation, a staging service and a separate production service with backups that restore. The work took 67 working days and 194 logged hours, all marked completed.

The work was also collaborative. The data model was reviewed with Member 1, the dispute hold and retry screen were joined with Member 2, tables and storage were shared with Member 4, and the money screens were built with Member 5 on top of the contract of 7 August. Each pair finished its two halves in neighbouring weeks, and that is why the team could run a tournament from funding to payout.

10.2 Lessons Learnt

- **Write the ledger first.** When every movement is a row, rounding, refunds and retries become questions that can be answered by reading.
- **Make repeats harmless.** A provider that can send an event twice needs a webhook that can receive it twice.
- **Fix the foundation early.** Fixing schema V1 in Week 2 let four other members build on it for the rest of the project.
- **Test the dull parts.** A restore test is not exciting, but it is the only proof that a backup is a backup.

- **Agree small interfaces early.** The contract of 7 August let two members build against the money module before it was finished.

10.3 Future Scope

- Obtain the payment partner's approval and run live payouts after the checklist of 16 September.
- Add an automatic reconciliation job between the provider's records and the ledger.
- Add a second payment route and a retry queue for failed webhooks.
- Move snapshots to object storage and take database backups to a second location.
- Run the smoke checks and auth tests automatically on every pull request.
- Extend the admin tools with identity checks for large prize pools, in line with the main report.

REFERENCES

[1] Oracle. Java SE 17 Documentation. https://docs.oracle.com/en/java/javase/17/

[2] VMware. Spring Boot 3.2 Reference Documentation. https://docs.spring.io/spring-boot/docs/3.2.0/reference/html/

[3] Oracle Corporation. MySQL 8.0 Reference Manual. https://dev.mysql.com/doc

[4] PingCAP. TiDB Cloud Documentation. https://docs.pingcap.com/tidbcloud

[5] Razorpay. Razorpay Payments Documentation, including Orders and Webhooks. https://razorpay.com/docs

[6] Render. Render Documentation: Web Services, Databases and Health Checks. https://render.com/docs

[7] OWASP Foundation. OWASP Top Ten, SQL Injection Prevention and Session Management Cheat Sheets. https://owasp.org

[8] Jones, M., Bradley, J., Sakimura, N. JSON Web Token (JWT), RFC 7519, IETF, 2015.

[9] HackAgon Project Report, Arya College of Engineering and IT, Jaipur, Session 2026-27.

[10] HackAgon Daily Work Log, 6 July 2026 to 6 October 2026, Members 1 to 5.

[11] HackAgon Team Modules and 12-Week Plan, 8 October 2026.

APPENDIX A: COMPLETE DAILY WORK LOG OF MEMBER 3

The table reproduces the daily work log of Member 3 (Money, Core, DevOps) for 6 July 2026 to 6 October 2026. It contains 67 entries and 194 hours. Every entry is marked Completed in the source log.

Table A.1: Daily work log of Member 3

Date	Hrs	Work description
06-07-2026	3	Compared MySQL host and PostgreSQL options
07-07-2026	2.5	Reviewed schema.sql against PRD tables
08-07-2026	4	Checked the secrets file and the target folder are in .gitignore
09-07-2026	3	Removed fake data from seed files
10-07-2026	2	Created Render staging service with health check
13-07-2026	3	Fixed schema V1 tables
14-07-2026	2.5	Finished migration runner

Date	Hrs	Work description
15-07-2026	4	Finished repo layer in RepoService.java
16-07-2026	3	Added the smoke-test script checks
17-07-2026	2	Set up automatic deploy to staging from main
20-07-2026	3	Added rate limits on login and OTP
21-07-2026	2.5	Added input validation
22-07-2026	4	Set CORS rules
23-07-2026	3	Added one error handler and request logs
24-07-2026	2	Wrote tests for auth routes
27-07-2026	3	Added repo functions for scoreboard queries
28-07-2026	2.5	Added database indexes
29-07-2026	4	Turned on backups
30-07-2026	3	Deployed Milestone 1 to staging
31-07-2026	2	Ran the test checklist and fixed issues
03-08-2026	3	Created tournaments and prize_splits tables
04-08-2026	2.5	Created payments table
05-08-2026	4	Created append-only wallet_ledger
06-08-2026	3	Opened Razorpay test account
07-08-2026	2	Wrote money API contract with Member 1 and 5
10-08-2026	3	Built Razorpay order for prize pool
11-08-2026	2.5	Built webhook with signature check
12-08-2026	4	Made webhook ignore repeated events
13-08-2026	3	Kept registration closed until pool is funded
14-08-2026	2	Tested host funding in test mode
17-08-2026	3	Built entry fee checkout
18-08-2026	2.5	Joined player to room after payment
19-08-2026	4	Built refund for cancelled tournaments
20-08-2026	3	Built refund when minimum players not met
21-08-2026	2	Prepared data for the rules page
24-08-2026	3	Built payout details with OTP to profile mobile
25-08-2026	2.5	Added Razorpay account validation
26-08-2026	4	Built payout job with 1% fee
27-08-2026	3	Split prize by rank and handled leftover paise
28-08-2026	2	Added ledger balance check
31-08-2026	3	Checked every route uses parameterized queries
01-09-2026	2.5	Tested SQL injection
02-09-2026	4	Tested session fixation and token misuse
03-09-2026	3	Fixed security findings
04-09-2026	2	Locked the make-admin script and the fix-admin script in production
07-09-2026	3	Wrote load simulation for 200 players polling one room
08-09-2026	2.5	Ran simulation up to 500 players
09-09-2026	4	Added database indexes from findings

Date	Hrs	Work description
10-09-2026	3	Added short cache for scoreboards
11-09-2026	2	Moved email from Gmail to a transactional provider
14-09-2026	3	Created separate production service
15-09-2026	2.5	Created separate production database
16-09-2026	4	Wrote Razorpay live-mode checklist
17-09-2026	3	Filed written confirmation about paid contests
18-09-2026	2	Ran backup restore test and set up error alerts
21-09-2026	3	Final pre-deploy checks
22-09-2026	2.5	Deployed to production
23-09-2026	4	Ran one small real tournament with friends
24-09-2026	3	Checked every payout
25-09-2026	2	Checked the ledger and held retro
28-09-2026	3	Checked the ledger after the first tournament
29-09-2026	2.5	Reviewed Razorpay webhook logs
30-09-2026	4	Fixed deploy and backup issues found
01-10-2026	3	Verified backups restore correctly
02-10-2026	2	Updated deployment notes
05-10-2026	3	Reviewed teammates' pull requests
06-10-2026	2.5	Final review and submission preparation

APPENDIX B: DATED JOINT WORK WITH OTHER MEMBERS

This appendix lists entries of Member 3 next to the matching entries of the other members (M1 to M5), as an aid to Chapter 6.

Table B.1: Dated joint work

Date	Member 3	Other members	Function
09 to 10-07	Removed fake data from seeds; created staging	M1 drafted data model (09-07), reviewed with M3 (10-07); M4 proctor API contract; M5 style guide	Week 1 decisions
13 to 15-07	Schema V1, migration runner, repo layer	M1 email and SMS OTP; M4 snapshot, violation and lock endpoints	Data layer
17-07	Automatic deploy to staging	M4 added duel and team tables with M3	Schema extension
20 to 24-07	Rate limits, validation, CORS, error handler, auth tests	M1 OTP and profile (13 to 17-07); M5 create-room design	Server protection
27 to 29-07	Scoreboard queries, indexes, backups	M5 polling scoreboard (29-07); M1 submission endpoint (29-07)	Scoreboard data
30 to 31-07	Milestone 1 deployed; test checklist	M1 tested a solo room (31-07); M5 fixed visual bugs (31-07)	Milestone 1
03 to 07-08	Money tables, Razorpay account, money contract (07-08)	M1 ranks and results data (03 to 04-08); M5 mistakes list; contract with M1 and M5	Money contract

Date	Member 3	Other members	Function
17 to 20-08	Entry fee, join after payment, refunds	M1 elimination, 18+ rule (17 to 20-08); M2 dispute form and payout block (17 to 18-08)	Entry fee and disputes
21-08	Data for the rules page	M5 rules page (25-08)	Rules page
24 to 25-08	Payout details, account validation	M2 failed-payouts screen and retry (24 to 25-08); M1 confirm and auto-confirm (24 to 25-08)	Payouts and results
26 to 28-08	Payout job, rank split, ledger check	M1 rank list (26-08), result locking test (27-08); M5 money screens (24 to 28-08); M2 admin tools test (28-08)	Payout
31-08 to 03-09	Security review and fixes	M1 organisation approval (31-08 to 02-09); M5 design pass (03 to 04-09)	Security
04-09	Locked the make-admin script and the fix-admin script in production	M2 protected the make-admin script and removed the fix-admin script (07-08)	Admin script security
07 to 10-09	Load simulation, indexes, scoreboard cache	M2 judge load test (07 to 09-09); M1 end-to-end script (07 to 09-09)	Load
14 to 17-09	Production service and database, checklist, written confirmation	M1 legal pages (14 to 17-09); M4 mock matches (14 to 16-09)	Production readiness
21 to 25-09	Final checks, deploy, first tournament, payout and ledger checks	M1 code freeze and watching (21 to 24-09); M4 monitoring (22 to 24-09); M2 disputes (24-09); M5 feedback (24-09); all retro (25-09)	First event
28 to 30-09	Ledger, webhook logs, deploy and backup fixes	M2 closed disputes (29-09); M4 tuned lock rules (29-09); M1 re-ran end-to-end test (30-09)	Follow-up
01 to 02-10	Backups verified; deployment notes	M1 updated api-contract.md (01-10); M4 checked snapshot job (02-10)	Closing